



Echo Oracle Audit Report

Prepared by [CodeHawks](#)

Version 2.0

Lead Eagles

[MoveJay](#)

[Incognito](#)

July 23, 2025

Contents

1	About CodeHawks	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	4
7.1	Medium Risk	4
7.1.1	Paired Asset Price Feed Integer Overflow	4
7.2	Low Risk	6
7.2.1	Redundant Initialization Functions and Risk of Orphaned/Incomplete State	6
7.2.2	Chainlink returns raw benchmark price, misinterpreting negative prices	6
7.2.3	Use of Deprecated and Inefficient <code>simple_map</code> Data Structure	7
7.3	Informational	8
7.3.1	Authorization Comment and Code Mismatch in Oracle Functions	8
7.3.2	Deprecated Authorization Function Retained	8
7.3.3	Redundant Error Constant	8
7.3.4	Typo in function name	9
7.3.5	Unnecessary Super Admin Check in View Function	9
7.3.6	Inefficient Use of <code>contains_key</code> and <code>add</code> Instead of <code>upsert</code> in <code>set_weight</code>	9
7.3.7	APPENDIX Formal Verification Results <code>chainlink.move</code>	10
7.4	Spec Overview for <code>echo_oracle::chainlink.move</code>	10
7.4.1	Redundant Conditional Logic in Price Calculation	10

1 About CodeHawks

[CodeHawks](#) is a leading competitive smart contract audit marketplace working to protect DeFi, protocols, their users, and funds from smart contract exploits through private and public smart contract security auditing competitions, joined by industry-leading auditors world-wide.

2 Disclaimer

The CodeHawks team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Echo Protocol is designed to enhance Bitcoin's utility within the Move ecosystem, particularly on the Aptos blockchain. It enables BTC holders to earn yield without selling their assets by bridging Bitcoin into the Move ecosystem through a token called aBTC, which is backed 1:1 by BTC.

Users can stake aBTC in Echo's multi-chain vaults, earning up to 30% APY, and participate in lending via Echo Lend without lockup periods. The protocol employs secure bridging mechanisms, including Hash-Timelock contracts and a decentralized validator network, to ensure transaction safety. Echo has garnered significant support from industry leaders and aims to provide sustainable yield opportunities for Bitcoin holders.

5 Audit Scope

The scope for this audit was limited to:

```
echo-core/echo-oracle/sources/chainlink.move
echo-core/echo-oracle/sources/oracle.move
echo-core/echo-oracle/sources/utils.move
echo-core/echo-acl/sources/acl_manage.move
```

6 Executive Summary

Over the course of 3 days, the CodeHawks team conducted an audit on the [Echo Oracle](#) smart contracts provided by [Echo Protocol](#). In this period, a total of 12 issues were found.

Summary

Project Name	Echo Oracle
Repository	echo-lending
Commit	2f473aa10f4f...
Audit Timeline	May 13th - May 15th, 2025
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	3
Informational	8
Gas Optimizations	0
Total Issues	12

Summary of Findings

[M-1] Paired Asset Price Feed Integer Overflow	Resolved
[L-1] Redundant Initialization Functions and Risk of Orphaned/Incomplete State	Acknowledged
[L-2] Chainlink returns raw benchmark price, misinterpreting negative prices	Acknowledged
[L-3] Use of Deprecated and Inefficient <code>simple_map</code> Data Structure	Acknowledged
[I-1] Authorization Comment and Code Mismatch in Oracle Functions	Acknowledged
[I-2] Deprecated Authorization Function Retained	Acknowledged
[I-3] Redundant Error Constant	Acknowledged
[I-4] Typo in function name	Resolved
[I-5] Unnecessary Super Admin Check in View Function	Resolved
[I-6] Inefficient Use of <code>contains_key</code> and <code>addInsteadofupsertinset_weight</code>	Resolved
[I-7] APPENDIX Formal Verification Results <code>chainlink.move</code>	Acknowledged
[I-8] Redundant Conditional Logic in Price Calculation	Resolved


```

202         aborts_if
↳ !simple_map::spec_contains_key(global<ChainlinkAssetPairMap>(@echo_oracle).value, asset) &&
203         get_base_decimal() - utils::get_base_decimal() <= 19;
    ~

        get_base_decimal() - utils::get_base_decimal() <= 19;
204     }
    ~
    .
256         (pow(10, pow_d) as u256)

```

Recommended Mitigation: The issue can be remediated through several architectural improvements to the decimal adjustment calculations:

1. Eliminate variable reassignment: Avoid modifying the same variable (`pow_d`) in conditional branches. Instead, calculate separate decimal adjustment factors for paired and unpaired assets to make the code more explicit and maintainable.
2. Use safe exponent ranges: Implement bounds checking for exponent values before passing them to the `pow` function. Since the function uses u64 arithmetic internally, exponents that would produce values larger than u64's maximum should be handled specially.
3. Consider implementing a safe power calculation that operates directly on u256 values for large exponents, bypassing the u64 limitation of the current `pow` function.

Echo: Fixed in commit [da4463a91fa5](#).

CodeHawks: Verified.

7.2 Low Risk

7.2.1 Redundant Initialization Functions and Risk of Orphaned/Incomplete State

Description: The `echo_oracle::oracle` module uses a private `init_module(publisher: &signer)` function, which automatically runs on first publication, placing key resources (`PythAssetPriceList`, `PythAssetPairMap`, `WeightMap`) under the publisher's account. The module also contains public entry functions `init_weight` and `init_asset_map`, callable only by `@echo_oracle`, which attempt to initialize `WeightMap` and `PythAssetPairMap` under `@echo_oracle` using `move_to`.

This creates two issues:

1. **Redundancy + Transaction Failures:** If `@echo_oracle` is the publisher, `init_module` correctly sets up all resources under it. Subsequent calls by `@echo_oracle` to `init_weight` or `init_asset_map` will then fail, as they attempt to `move_to` resources that already exist. These public functions become redundant and error-prone.
2. **Orphaned Resources & Incomplete Oracle State:** If an account other than `@echo_oracle` publishes the module, `init_module` places the resources under that other account (orphaned from the oracle's perspective). If `@echo_oracle` then calls `init_weight` and `init_asset_map`, it will create `WeightMap` and `PythAssetPairMap` under its own address. However, `PythAssetPriceList` (also initialized by `init_module`) would remain with the other publisher, leaving `@echo_oracle` with an incomplete and non-functional set of resources, as the oracle's core logic expects all three under `@echo_oracle`.

Impact:

- Legitimate calls to `init_weight` or `init_asset_map` by `@echo_oracle` (if it was the publisher) will fail, causing confusion and potentially blocking administrative tasks if these functions are misunderstood to be re-initializers.
- If the module is not published by `@echo_oracle`, the primary resources are orphaned. Even if `@echo_oracle` then uses `init_weight` and `init_asset_map`, the oracle remains non-functional due to missing `PythAssetPriceList` under its account, leading to failures in core oracle operations.
- The redundant functions make the module harder to understand and maintain.

Recommended Mitigation: To address these issues, a two-pronged approach is recommended for remediation.

Primary Initialization via `init_module`:

- Strictly rely on the private `fun init_module(account: &signer)` for the complete, one-time initialization of all necessary resources.
- Ensure the module is published by the `@echo_oracle` account. This guarantees `init_module` places resources correctly under `@echo_oracle`.

Remove Redundant Public Functions:

- Delete public entry `fun init_weight(account: &signer).`
- Delete public entry `fun init_asset_map(account: &signer).`

Their functionality is either covered by `init_module` (if `@echo_oracle` publishes) or leads to incomplete states/errors.

Echo: Acknowledged.

7.2.2 Chainlink returns raw benchmark price, misinterpreting negative prices

Description: The Echo Oracle's `get_asset_price` and `batch_set_asset_price` functions in `chainlink.move` fetch prices from Chainlink's router benchmark data. This benchmark is extracted from the raw report data and its price, which is a signed `i192`. However, the `i192` is treated and returned as a raw `u256` by Chainlink:

The Echo Oracle does not perform any validation on this returned `u256` value. This means a negative price from Chainlink would be misinterpreted as a very large positive number, which could lead to having near-unlimited collateral value to borrow against in the Echo lending protocol. The issue is particularly concerning in functions like `get_user_account_data` in `user_logic.move` and `calculate_available_borrows` in `generic_logic.move`, where these potentially inflated prices would directly affect collateral calculations and borrowing power.

Recommended Mitigation: Add validation in the price fetching and setting functions to ensure prices are within valid ranges:

This validation should be added to both `get_asset_price` and `batch_set_asset_price` functions to ensure:

- Negative prices (which would appear as values $> I192_MAX$) are rejected
- Zero prices are rejected to prevent division by zero errors in health factor computations
- Only valid positive prices within the `i192` range are accepted

Echo: Acknowledged.

7.2.3 Use of Deprecated and Inefficient `simple_map` Data Structure

Description: The project uses the `simple_map` module from `aptos_std` to manage key-value mappings. However, the original implementation is explicitly marked as **deprecated** in the Aptos core libraries due to performance limitations. According to the module documentation:

- Key lookups, additions, and removals operate in **$O(N)$** time.
- The keys are stored in an **unsorted vector**, resulting in inefficient operations for large datasets.

Impact: This design choice can lead to scalability issues and poor runtime performance, especially in contracts that frequently modify or query the map.

Proof of Concept: https://github.com/aptos-labs/aptos-core/blob/main/aptos-move/framework/aptos-stdlib/sources/simple_map.move#L8-L9

```
/// DEPRECATED: since it's implementation is inneficient, it
/// has been deprecated in favor of `ordered_map.move`.
```

Recommended Mitigation: Migrate from `simple_map` to the recommended `ordered_map.move` module or another efficient map implementation that provides $O(\log N)$ or $O(1)$ operations for lookups and updates.

Echo: Acknowledged.

7.3 Informational

7.3.1 Authorization Comment and Code Mismatch in Oracle Functions

Description: The authorization checks that should restrict price setting capabilities have been commented out in both oracle implementations (Pyth and Chainlink). This creates ambiguity about whether these functions are intentionally accessible to all users or if this is an oversight.

<https://github.com/echo-protocol/echo-lending/blob/audit/MoveJay/echo-core/echo-oracle/sources/chainlink.move#L236-L246>

<https://github.com/echo-protocol/echo-lending/blob/audit/MoveJay/echo-core/echo-oracle/sources/oracle.move#L301-L307>

Recommended Mitigation: Option 1: If these functions should be admin-restricted :

- Uncomment the admin check lines to match the stated intent in the comments
- This would align with the security model of other functions in the codebase

Option 2: If these functions are intentionally open to all users:

- Update the comments to remove the misleading "ensure only admins can call" text
- Add new comments explaining why price setting is allowed for all users

Echo: Acknowledged.

7.3.2 Deprecated Authorization Function Retained

Description: The codebase contains a deprecated function `check_is_asset_listing_or_pool_admin` that has been marked as moved to the `utils` module. Keeping deprecated code in the codebase can lead to confusion, potential misuse, and maintenance overhead. The comment indicates this function has been migrated, yet the implementation remains in place.

<https://github.com/echo-protocol/echo-lending/blob/main/echo-core/echo-oracle/sources/oracle.move#L95-L99>

Recommended Mitigation: Remove the deprecated function entirely since it has been moved to the `utils` module

Echo: Acknowledged.

7.3.3 Redundant Error Constant

Description: The codebase contains a contradictory situation with the `NOT_ASSET_LISTING_OR_POOL_ADMIN` error constant. The constant is marked as deprecated with a comment indicating it has been moved to the `utils` module, yet it is actively used within the current file. This creates confusion about whether the constant should be used or not, and contradicts the deprecation notice. Such inconsistencies between comments and actual code usage can lead to maintenance issues and developer confusion.

<https://github.com/echo-protocol/echo-lending/blob/main/echo-core/echo-oracle/sources/oracle.move#L26-L27>

Recommended Mitigation: Choose one of these approaches:

- If the constant should be deprecated:
- Remove the constant from this file
- Update all usages to reference the constant from the `utils` module
- Remove the misleading comments

If the constant should remain in this file:

- Remove the deprecation comment
- Update the documentation to clearly indicate this is the active version

- Consider adding a comment explaining why this constant is maintained locally

Echo: Acknowledged.

7.3.4 Typo in function name

Description: The function `get_base_decimial` is consistently misspelled across multiple files. The intended name is likely `get_base_decimal`. This misspelling occurs in:

- `echo-core/echo-oracle/sources/oracle.move`
- `echo-core/echo-oracle/sources/chainlink.move`
- `echo-core/echo-oracle/sources/utls.move`

Recommended Mitigation: Rename the function `get_base_decimial` to `get_base_decimal` in all its declarations and usages within the affected files.

Echo: Fixed in commit [da4463a91fa5](#).

CodeHawks: Verified.

7.3.5 Unnecessary Super Admin Check in View Function

Description: The `get_role` function is declared as a `#[view]` function, indicating that it is read-only and does not mutate state. Despite this, it includes a `check_super_admin(user)` call and accepts a user address parameter that is only used for this permission check. This increases the complexity as the function merely returns data from storage without performing any privileged or sensitive operations

Impact: Requiring super admin access introduces unnecessary complexity and access restrictions.

Proof of Concept:

Recommended Mitigation:

- Remove super admin check

```
#[view]
- public fun get_role(role: String, user: address): vector<address> acquires Role {
+ public fun get_role(role: String): vector<address> acquires Role {
-     check_super_admin(user);
    let role_res = borrow_global<Role>(@echo_acl);

    *smart_table::borrow(&role_res.acl_instance, role)
}
```

Echo: Fixed in commit [da4463a91fa5](#).

CodeHawks: Verified.

7.3.6 Inefficient Use of `contains_key` and `add` Instead of `upsert` in `set_weight`

Description: The `set_weight` function currently checks for key existence using `simple_map::contains_key` before deciding to update or insert the value. This results in two sequential $O(N)$ operations:

- `contains_key` performs a linear search to find the key, and
- `add` also performs a redundant `find` internally before inserting.

This leads to unnecessary double iteration over the `SimpleMap` data. In contrast, using an `upsert`-style operation (which performs a single pass to either insert or update the key-value pair) would achieve the same result more efficiently, in a single $O(N)$ operation.

Recommended Mitigation: Refactor `set_weight` to replace the `contains_key` + `add` logic with a single-pass `upsert` function.

Echo: Fixed in commit [da4463a91fa5](#).

CodeHawks: Verified.

7.3.7 APPENDIX Formal Verification Results `chainlink.move`

7.4 Spec Overview for `echo_oracle::chainlink.move`

Functionality	Details
function get_asset_price	
Preconditions	Explicitly partial specification with selected abort conditions.
Abort Conditions	Aborts if price is invalid (E_INVALID_PRICE).
Abort Conditions	Aborts if ChainlinkAssetPriceList or ChainlinkAssetPairMap resources don't exist.
Abort Conditions	Aborts if requested asset doesn't exist (E_ASSET_NOT_EXISTS).
Abort Conditions	For paired assets, aborts if base asset doesn't exist.
Abort Conditions	Aborts if final calculated price is zero (E_INVALID_PRICE).
Abort Conditions	Aborts when pow calculation should be safe for paired assets (base_decimal > 0).
Abort Conditions	Aborts when pow calculation should be safe for non-paired assets (base_decimal < 0).
function get_asset_price_and_update_time ‘	
Preconditions	Explicitly partial specification (pragma aborts_if_is_partial = true).
Postconditions	Returns (0, 0) if ChainlinkAssetPriceList resource doesn't exist.
Postconditions	Returns (0, 0) if ChainlinkAssetPairMap resource doesn't exist.
Postconditions	Returns (0, 0) if requested asset doesn't exist.
Postconditions	For non-paired assets, returns correct price using price_for_regular_asset helper.
Postconditions	For paired assets, returns correct price using price_for_paired_asset helper.
function validate_price_deviation ‘	
Postconditions	If old_price is 0, always accepts new price.
Postconditions	If prices are equal, no deviation (returns true).
Postconditions	If deviation is within MAX_PRICE_DEVIATION_BPS, returns true.
Postconditions	If deviation is too large, returns false.

Echo: Acknowledged.

7.4.1 Redundant Conditional Logic in Price Calculation

Description: Multiple price calculation functions in both `oracle.move` and `chainlink.move` contain two redundant patterns when calculating divisor values:

1. Redundant Direct/Paired Asset Branching: In the else branch (direct asset case), `pow_d` is always 0 because it's only modified in the if branch (paired asset case), making the condition check unnecessary.
2. Redundant Zero Check: The pattern `if (pow_d > 0) {...} else {1u256}` is redundant because the `pow` function already handles this case internally. The `pow` function in `aptos_std::math64` has a built-in check that returns 1 when the exponent is zero:

The issue is present in four locations:

<https://github.com/echo-proto/echo-lending/blob/main/echo-core/echo-oracle/sources/chainlink.move#L204-L210>

<https://github.com/echo-proto/echo-lending/blob/main/echo-core/echo-oracle/sources/chainlink.move#L160-L171>

<https://github.com/echo-proto/echo-lending/blob/main/echo-core/echo-oracle/sources/oracle.move#L269-L275>

Recommended Mitigation: Option 1: Simplify direct asset case

Ex:

Option 2 (preferred): Remove redundant conditional entirely

Echo: Fixed in commit [da4463a91fa5](#).

CodeHawks: Verified.